Hadoop Streaming Practice in Google Colab — Fixed

This notebook uses a **real Hadoop environment** in Colab and **does not use PySpark**.

It fixes the earlier issues:

- correct Hadoop download and install
- correct `PATH` / `JAVA_HOME` / `HADOOP_HOME`
- correct **streaming JAR path**
- uses **Python** `subprocess` instead of fragile notebook shell magics
- uses **local file system mode** with `file:///...`
- reads `part-00000` **only after** the Hadoop job succeeds

You do **not** need any parquet file for this practice. Hadoop Streaming is designed for **plain text** input.

```
1
2 # Step 1: Install Java + Hadoop correctly
3 import os
4 import subprocess
5 from pathlib import Path
6
7 HADOOP_VERSION = "3.3.6"
8 HADOOP_DIR = f"/content/hadoop-{HADOOP_VERSION}"
9 HADOOP_TAR = f"/content/hadoop-{HADOOP_VERSION}.tar.gz"
10 HADOOP_URL = f"https://archive.apache.org/dist/hadoop/common/hadoop-{HADOOP_VERSION}/hadoop-{HAD(
11
12 def run(cmd, env=None):
13     print("RUN:", cmd)
14     completed = subprocess.run(
15         cmd,
16         shell=True,
17         text=True,
18         stdout=subprocess.PIPE,
```

```
        stderr=subprocess.STDOUT,
        env=env,
    )
    print(completed.stdout)
    if completed.returncode != 0:
        raise RuntimeError(f"Command failed with exit code {completed.returncode}: {cmd}")
    return completed

run("apt-get -qq update")
run("apt-get install -y openjdk-11-jdk-headless wget tar > /dev/null")

if not Path(HADOOP_DIR).exists():
    run(f"wget -q -O {HADOOP_TAR} {HADOOP_URL}")
    run(f"tar -xzf {HADOOP_TAR} -C /content/")

os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-11-openjdk-amd64"
os.environ["HADOOP_HOME"] = HADOOP_DIR
os.environ["HADOOP_CONF_DIR"] = f"{HADOOP_DIR}/etc/hadoop"
os.environ["PATH"] = os.environ["PATH"] + f":{HADOOP_DIR}/bin:{HADOOP_DIR}/sbin"

ENV = os.environ.copy()

print("JAVA_HOME =", os.environ["JAVA_HOME"])
print("HADOOP_HOME =", os.environ["HADOOP_HOME"])

run("java -version", env=ENV)
run(f"{HADOOP_DIR}/bin/hadoop version", env=ENV)

```

```
RUN: apt-get -qq update
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois

RUN: apt-get install -y openjdk-11-jdk-headless wget tar > /dev/null

RUN: wget -q -O /content/hadoop-3.3.6.tar.gz https://archive.apache.org/dist/hadoop/common/hadoop-3.3

RUN: tar -xzf /content/hadoop-3.3.6.tar.gz -C /content/

JAVA_HOME = /usr/lib/jvm/java-11-openjdk-amd64
HADOOP_HOME = /content/hadoop-3.3.6
```

```
RUN: java -version
openjdk version "17.0.18" 2026-01-20
OpenJDK Runtime Environment (build 17.0.18+8-Ubuntu-122.04.1)
OpenJDK 64-Bit Server VM (build 17.0.18+8-Ubuntu-122.04.1, mixed mode, sharing)

RUN: /content/hadoop-3.3.6/bin/hadoop version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be78238728da9266a4f88195058f08fd012bf9
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1
From source with checksum 5652179ad55f76cb287d9c633bb53bbd
This command was run using /content/hadoop-3.3.6/share/hadoop/common/hadoop-common-3.3.6.jar

CompletedProcess(args='/content/hadoop-3.3.6/bin/hadoop version', returncode=0, stdout='Hadoop
3.3.6\nSource code repository https://github.com/apache/hadoop.git -r
1be78238728da9266a4f88195058f08fd012bf9c\nCompiled by ubuntu on 2023-06-18T08:22Z\nCompiled on
platform linux-x86_64\nCompiled with protoc 3.7.1\nFrom source with checksum
5652179ad55f76cb287d9c633bb53bbd\nThis command was run using /content/hadoop-
3.3.6/share/hadoop/common/hadoop-common-3.3.6.jar\n')
```

```
1
2 # Step 2: Create sample input data
3 from pathlib import Path
4
5 sample_dir = Path("/content/hadoop_input")
6 sample_dir.mkdir(exist_ok=True)
7
8 sample_text_1 = '''2026-04-22 09:36:39 INFO BlockStateChange replication started successfully
9 2026-04-22 09:37:10 WARN DataNode slow response from storage subsystem
10 2026-04-22 09:37:55 ERROR NameNode failed to process metadata synchronization
11 2026-04-22 09:38:15 INFO ReplicationManager completed rebalancing operation
12 2026-04-22 09:38:45 INFO authenticationservice accepted token request
13 '''
14
15 sample_text_2 = '''2026-04-22 10:01:03 INFO BlockManager received heartbeat from worker node
16 2026-04-22 10:02:28 WARN CheckpointCoordinator detected delayed checkpoint creation
17 2026-04-22 10:03:44 INFO metadata synchronization finished without exception
18 2026-04-22 10:04:59 ERROR configurationvalidationservice rejected invalid property
```

```
2026-04-22 10:05:31 INFO replicationmonitor completed verification successfully
'''

(sample_dir / "log1.txt").write_text(sample_text_1)
(sample_dir / "log2.txt").write_text(sample_text_2)

run("ls -R /content/hadoop_input")
run('echo "----- log1.txt -----" && cat /content/hadoop_input/log1.txt')
run('echo "----- log2.txt -----" && cat /content/hadoop_input/log2.txt')

```

```
RUN: ls -R /content/hadoop_input
/content/hadoop_input:
log1.txt
log2.txt

RUN: echo "----- log1.txt -----" && cat /content/hadoop_input/log1.txt
----- log1.txt -----
2026-04-22 09:36:39 INFO BlockStateChange replication started successfully
2026-04-22 09:37:10 WARN DataNode slow response from storage subsystem
2026-04-22 09:37:55 ERROR NameNode failed to process metadata synchronization
2026-04-22 09:38:15 INFO ReplicationManager completed rebalancing operation
2026-04-22 09:38:45 INFO authenticationservice accepted token request

RUN: echo "----- log2.txt -----" && cat /content/hadoop_input/log2.txt
----- log2.txt -----
2026-04-22 10:01:03 INFO BlockManager received heartbeat from worker node
2026-04-22 10:02:28 WARN CheckpointCoordinator detected delayed checkpoint creation
2026-04-22 10:03:44 INFO metadata synchronization finished without exception
2026-04-22 10:04:59 ERROR configurationvalidationservice rejected invalid property
2026-04-22 10:05:31 INFO replicationmonitor completed verification successfully

CompletedProcess(args='echo "----- log2.txt -----" && cat /content/hadoop_input/log2.txt',
returncode=0, stdout='----- log2.txt -----\n2026-04-22 10:01:03 INFO BlockManager received heartbeat
from worker node\n2026-04-22 10:02:28 WARN CheckpointCoordinator detected delayed checkpoint
creation\n2026-04-22 10:03:44 INFO metadata synchronization finished without exception\n2026-04-22
10:04:59 ERROR configurationvalidationservice rejected invalid property\n2026-04-22 10:05:31 INFO
replicationmonitor completed verification successfully\n')
```

```

# Step 3: Locate Hadoop example and streaming JAR files
import glob

example_matches = glob.glob(
    f"{os.environ['HADOOP_HOME']}/share/hadoop/mapreduce/hadoop-mapreduce-examples-*.jar"
)
streaming_matches = glob.glob(
    f"{os.environ['HADOOP_HOME']}/share/hadoop/tools/lib/hadoop-streaming-*.jar"
)

assert example_matches, "Could not find Hadoop examples JAR."
assert streaming_matches, "Could not find Hadoop streaming JAR."

EXAMPLE_JAR = example_matches[0]
STREAMING_JAR = streaming_matches[0]

print("Example JAR  :", EXAMPLE_JAR)
print("Streaming JAR:", STREAMING_JAR)

```

```
Example JAR  : /content/hadoop-3.3.6/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar
Streaming JAR: /content/hadoop-3.3.6/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar
```

```

# Step 4: Run a built-in Hadoop example (grep) in LOCAL mode
run("rm -rf /content/hadoop_grep_output")

grep_cmd = (
    f'"{os.environ["HADOOP_HOME"]}/bin/hadoop" jar "{EXAMPLE_JAR}" grep '
    f'-D mapreduce.framework.name=local '
    f'-D fs.defaultFS=file:/// '
    f'file:///content/hadoop_input '
    f'file:///content/hadoop_grep_output '
    f'"INFO|ERROR"'
)
run(grep_cmd, env=ENV)

```

```
run('echo "----- grep output -----" && cat /content/hadoop_grep_output/*')

```

```
                BAD_ID=0
                CONNECTION=0
                IO_ERROR=0
                WRONG_LENGTH=0
                WRONG_MAP=0
                WRONG_REDUCE=0
        File Input Format Counters
                Bytes Read=141
        File Output Format Counters
                Bytes Written=27

RUN: echo "----- grep output -----" && cat /content/hadoop_grep_output/*
----- grep output -----
6       INFO
2       ERROR

CompletedProcess(args='echo "----- grep output -----" && cat /content/hadoop_grep_output/*',
returncode=0, stdout='----- grep output -----\n6\tINFO\n2\tERROR\n')
```

```
 1
 2 # Step 5: Create mapper and reducer scripts for lowercase word count
 3 import textwrap
 4 from pathlib import Path
 5
 6 mapper_wc = textwrap.dedent("""#!/usr/bin/env python3
 7 import sys
 8 import re
 9
10 for line in sys.stdin:
11     for word in re.findall(r"[A-Za-z0-9_]+", line.lower()):
12         print(f"{word}\t1")
13 """)
14
15 reducer_sum = textwrap.dedent("""#!/usr/bin/env python3
16 import sys
17
18 current_key = None
19 current_total = 0
20
21 for line in sys.stdin:
```

```
    line = line.strip()
    if not line:
        continue
    key, value = line.split("\t", 1)
    value = int(value)

    if key == current_key:
        current_total += value
    else:
        if current_key is not None:
            print(f"{current_key}\t{current_total}")
        current_key = key
        current_total = value

if current_key is not None:
    print(f"{current_key}\t{current_total}")
""")

Path("/content/mapper_wc.py").write_text(mapper_wc)
Path("/content/reducer_sum.py").write_text(reducer_sum)

run("chmod +x /content/mapper_wc.py /content/reducer_sum.py")
run("sed -n '1,120p' /content/mapper_wc.py")
run('echo "-----" && sed -n "1,160p" /content/reducer_sum.py')

```

```
RUN: chmod +x /content/mapper_wc.py /content/reducer_sum.py

RUN: sed -n '1,120p' /content/mapper_wc.py
#!/usr/bin/env python3
import sys
import re

for line in sys.stdin:
    for word in re.findall(r"[A-Za-z0-9_]+", line.lower()):
        print(f"{word}  1")

RUN: echo "-----" && sed -n "1,160p" /content/reducer_sum.py
-----
#!/usr/bin/env python3
```

```
import sys

current_key = None
current_total = 0

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    key, value = line.split("   ", 1)
    value = int(value)

    if key == current_key:
        current_total += value
    else:
        if current_key is not None:
            print(f"{current_key}       {current_total}")
        current_key = key
        current_total = value

if current_key is not None:
    print(f"{current_key}       {current_total}")
```

```
CompletedProcess(args='echo "-----" && sed -n "1,160p" /content/reducer_sum.py', returncode=0,
stdout='-----\n#!/usr/bin/env python3\nimport sys\n\ncurrent_key = None\ncurrent_total = 0\n\nfor
line in sys.stdin:\n    line = line.strip()\n    if not line:\n        continue\n    key, value =
line.split("\t", 1)\n    value = int(value)\n\n    if key == current_key:\n        current_total +=
value\n    else:\n        if current_key is not None:\n            print(f"
{current_key}\t{current_total}")\n        current_key = key\n        current_total = value\n\nif
current_key is not None:\n    print(f"{current_key}\t{current_total}")\n')
```

```
1
2 # Step 6: Run Hadoop Streaming word count in LOCAL mode
3 run("rm -rf /content/hadoop_wc_output")
4
5 wc_cmd = (
6     f'"{os.environ["HADOOP_HOME"]}/bin/hadoop" jar "{STREAMING_JAR}" '
7     f'-D mapreduce.framework.name=local '
8     f'-D fs.defaultFS=file:/// '
9     f'-files /content/mapper_wc.py,/content/reducer_sum.py '
10    f'-mapper "python3 mapper_wc.py" '
```

```
    f'-reducer "python3 reducer_sum.py" '
    f'-input file:///content/hadoop_input '
    f'-output file:///content/hadoop_wc_output'
)
run(wc_cmd, env=ENV)

run('echo "----- first 30 word-count results -----" && head -30 /content/hadoop_wc_output/part-00
```

```
28      1
31      1
36      1
37      2
38      2
39      1
44      1
45      1
55      1
59      1
accepted        1
authenticationservice   1
blockmanager    1
blockstatechange        1
checkpoint      1
checkpointcoordinator   1
completed       2
configurationvalidationservice  1
creation        1
datanode        1

CompletedProcess(args='echo "----- first 30 word-count results -----" && head -30
/content/hadoop_wc_output/part-00000', returncode=0, stdout='----- first 30 word-count results ----
-
\n01\t1\n02\t1\n03\t2\n04\t11\n05\t1\n09\t5\n10\t6\n15\t1\n2026\t10\n22\t10\n28\t1\n31\t1\n36\t1\n37
```

```
1
2 # Step 7: Quick checks on word-count output
3 run('echo "Total unique words:" && wc -l /content/hadoop_wc_output/part-00000')
4 run('echo "" && echo "Top words by frequency:" && sort -k2,2nr /content/hadoop_wc_output/part-00(
5
```

```
RUN: echo "Total unique words:" && wc -l /content/hadoop_wc_output/part-00000
Total unique words:
66 /content/hadoop_wc_output/part-00000

RUN: echo "" && echo "Top words by frequency:" && sort -k2,2nr /content/hadoop_wc_output/part-00000 |

Top words by frequency:
04      11
2026    10
```

```
22      10
10      6
info    6
09      5
03      2
37      2
38      2
completed       2
error   2
from    2
metadata        2
successfully    2
synchronization 2

CompletedProcess(args='echo "" && echo "Top words by frequency:" && sort -k2,2nr
/content/hadoop_wc_output/part-00000 | head -15', returncode=0, stdout='\nTop words by
frequency:\n04\t11\n2026\t10\n22\t10\n10\t6\ninfo\t6\n09\t5\n03\t2\n37\t2\n38\t2\ncompleted\t2\nerror'
```

```
 1
 2 # Step 8: Create mapper and reducer for distinct words
 3 mapper_distinct = textwrap.dedent("""#!/usr/bin/env python3
 4 import sys
 5 import re
 6
 7 for line in sys.stdin:
 8     for word in re.findall(r"[A-Za-z0-9_]+", line.lower()):
 9         print(word)
10 """)
11
12 reducer_distinct = textwrap.dedent("""#!/usr/bin/env python3
13 import sys
14
15 previous = None
16
17 for line in sys.stdin:
18     word = line.strip()
19     if not word:
20         continue
21     if word != previous:
```

```
        print(word)
        previous = word
""")

Path("/content/mapper_distinct.py").write_text(mapper_distinct)
Path("/content/reducer_distinct.py").write_text(reducer_distinct)

run("chmod +x /content/mapper_distinct.py /content/reducer_distinct.py")

```

```
RUN: chmod +x /content/mapper_distinct.py /content/reducer_distinct.py

CompletedProcess(args='chmod +x /content/mapper_distinct.py /content/reducer_distinct.py',
returncode=0, stdout='')
```

```

# Step 9: Run Hadoop Streaming distinct-word job
run("rm -rf /content/hadoop_distinct_output")

distinct_cmd = (
    f'"{os.environ["HADOOP_HOME"]}/bin/hadoop" jar "{STREAMING_JAR}" '
    f'-D mapreduce.framework.name=local '
    f'-D fs.defaultFS=file:/// '
    f'-files /content/mapper_distinct.py,/content/reducer_distinct.py '
    f'-mapper "python3 mapper_distinct.py" '
    f'-reducer "python3 reducer_distinct.py" '
    f'-input file:///content/hadoop_input '
    f'-output file:///content/hadoop_distinct_output'
)
run(distinct_cmd, env=ENV)

run('echo "----- first 30 distinct words -----" && head -30 /content/hadoop_distinct_output/part-
run('echo "" && echo "Distinct word count:" && wc -l /content/hadoop_distinct_output/part-00000')

```

```
creation
datanode

RUN: echo "" && echo "Distinct word count:" && wc -l /content/hadoop_distinct_output/part-00000

Distinct word count:
66 /content/hadoop_distinct_output/part-00000

CompletedProcess(args='echo "" && echo "Distinct word count:" && wc -l
/content/hadoop_distinct_output/part-00000', returncode=0, stdout='\nDistinct word count:\n66
```

```
# Step 10: Create mapper for long words (> 12 characters)
mapper_long = textwrap.dedent("""#!/usr/bin/env python3
import sys
import re

for line in sys.stdin:
    for word in re.findall(r"[A-Za-z0-9_]+", line.lower()):
        if len(word) > 12:
            print(f"{word}\t1")
""")

Path("/content/mapper_long.py").write_text(mapper_long)
run("chmod +x /content/mapper_long.py")
run("sed -n '1,120p' /content/mapper_long.py")

```

```
RUN: chmod +x /content/mapper_long.py

RUN: sed -n '1,120p' /content/mapper_long.py
#!/usr/bin/env python3
import sys
import re

for line in sys.stdin:
    for word in re.findall(r"[A-Za-z0-9_]+", line.lower()):
        if len(word) > 12:
            print(f"{word}      1")
```

```
CompletedProcess(args="sed -n '1,120p' /content/mapper_long.py", returncode=0, stdout='#!/usr/bin/env
python3\nimport sys\nimport re\n\nfor line in sys.stdin:\n    for word in re.findall(r"[A-Za-z0-
9_]+", line.lower()):\n        if len(word) > 12:\n            print(f"{word}\t1")\n')
```

```
 1
 2 # Step 11: Run Hadoop Streaming long-word job
 3 run("rm -rf /content/hadoop_longwords_output")
 4
 5 long_cmd = (
 6     f'"{os.environ["HADOOP_HOME"]}/bin/hadoop" jar "{STREAMING_JAR}" '
 7     f'-D mapreduce.framework.name=local '
 8     f'-D fs.defaultFS=file:/// '
 9     f'-files /content/mapper_long.py,/content/reducer_sum.py '
10     f'-mapper "python3 mapper_long.py" '
11     f'-reducer "python3 reducer_sum.py" '
12     f'-input file:///content/hadoop_input '
13     f'-output file:///content/hadoop_longwords_output'
14 )
15 run(long_cmd, env=ENV)
16
17 run('echo "----- long words and their counts -----" && cat /content/hadoop_longwords_output/part-
18
```

```
RUN: rm -rf /content/hadoop_longwords_output

RUN: "/content/hadoop-3.3.6/bin/hadoop" jar "/content/hadoop-3.3.6/share/hadoop/tools/lib/hadoop-str
2026-04-23 05:27:25,170 INFO impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2026-04-23 05:27:25,313 INFO impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s
2026-04-23 05:27:25,313 INFO impl.MetricsSystemImpl: JobTracker metrics system started
2026-04-23 05:27:25,329 WARN impl.MetricsSystemImpl: JobTracker metrics system already initialized!
2026-04-23 05:27:25,536 INFO mapred.FileInputFormat: Total input files to process : 2
2026-04-23 05:27:25,552 INFO mapreduce.JobSubmitter: number of splits:2
2026-04-23 05:27:25,749 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_local904016510_0
2026-04-23 05:27:25,749 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-04-23 05:27:26,078 INFO mapred.LocalDistributedCacheManager: Localized file:/content/mapper_lon
2026-04-23 05:27:26,103 INFO mapred.LocalDistributedCacheManager: Localized file:/content/reducer_su
2026-04-23 05:27:26,240 INFO mapreduce.Job: The url to track the job: http://localhost:8080/
2026-04-23 05:27:26,244 INFO mapreduce.Job: Running job: job_local904016510_0001
2026-04-23 05:27:26,245 INFO mapred.LocalJobRunner: OutputCommitter set in config null
```

```
2026-04-23 05:27:26,248 INFO mapred.LocalJobRunner: OutputCommitter is org.apache.hadoop.mapred.File
2026-04-23 05:27:26,256 INFO output.FileOutputCommitter: File Output Committer Algorithm version is
2026-04-23 05:27:26,256 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary
2026-04-23 05:27:26,317 INFO mapred.LocalJobRunner: Waiting for map tasks
2026-04-23 05:27:26,321 INFO mapred.LocalJobRunner: Starting task: attempt_local904016510_0001_m_000
2026-04-23 05:27:26,354 INFO output.FileOutputCommitter: File Output Committer Algorithm version is
2026-04-23 05:27:26,356 INFO output.FileOutputCommitter: FileOutputCommitter skip cleanup _temporary
2026-04-23 05:27:26,374 INFO mapred.Task:  Using ResourceCalculatorProcessTree : [ ]
2026-04-23 05:27:26,384 INFO mapred.MapTask: Processing split: file:/content/hadoop_input/log2.txt:0
2026-04-23 05:27:26,397 INFO mapred.MapTask: numReduceTasks: 1
2026-04-23 05:27:26,436 INFO mapred.MapTask: (EQUATOR) 0 kvi 26214396(104857584)
2026-04-23 05:27:26,436 INFO mapred.MapTask: mapreduce.task.io.sort.mb: 100
2026-04-23 05:27:26,436 INFO mapred.MapTask: soft limit at 83886080
2026-04-23 05:27:26,436 INFO mapred.MapTask: bufstart = 0; bufvoid = 104857600
2026-04-23 05:27:26,436 INFO mapred.MapTask: kvstart = 26214396; length = 6553600
2026-04-23 05:27:26,440 INFO mapred.MapTask: Map output collector class = org.apache.hadoop.mapred.M
2026-04-23 05:27:26,446 INFO streaming.PipeMapRed: PipeMapRed exec [/usr/bin/python3, mapper_long.py
2026-04-23 05:27:26,451 INFO Configuration.deprecation: mapred.work.output.dir is deprecated. Instea
2026-04-23 05:27:26,454 INFO Configuration.deprecation: mapred.local.dir is deprecated. Instead, use
2026-04-23 05:27:26,454 INFO Configuration.deprecation: map.input.file is deprecated. Instead, use m
2026-04-23 05:27:26,455 INFO Configuration.deprecation: map.input.length is deprecated. Instead, use
2026-04-23 05:27:26,456 INFO Configuration.deprecation: mapred.job.id is deprecated. Instead, use ma
2026-04-23 05:27:26,456 INFO Configuration.deprecation: mapred.task.partition is deprecated. Instead
2026-04-23 05:27:26,457 INFO Configuration.deprecation: map.input.start is deprecated. Instead, use
2026-04-23 05:27:26,457 INFO Configuration.deprecation: mapred.task.is.map is deprecated. Instead, u
2026-04-23 05:27:26,458 INFO Configuration.deprecation: mapred.task.id is deprecated. Instead, use m
2026-04-23 05:27:26,461 INFO Configuration.deprecation: mapred.tip.id is deprecated. Instead, use ma
2026-04-23 05:27:26,461 INFO Configuration.deprecation: mapred.skip.on is deprecated. Instead, use m
2026-04-23 05:27:26,462 INFO Configuration.deprecation: user.name is deprecated. Instead, use mapred
2026-04-23 05:27:26,492 INFO streaming.PipeMapRed: R/W/S=1/0/0 in:NA [rec/s] out:NA [rec/s]
2026-04-23 05:27:26,508 INFO streaming.PipeMapRed: Records R/W=5/1
2026-04-23 05:27:26,509 INFO streaming.PipeMapRed: MRErrorThread done
2026-04-23 05:27:26,510 INFO streaming.PipeMapRed: mapRedFinished
2026-04-23 05:27:26,513 INFO mapred.LocalJobRunner:
2026-04-23 05:27:26,513 INFO mapred.MapTask: Starting flush of map output
2026-04-23 05:27:26,513 INFO mapred.MapTask: Spilling map output
2026-04-23 05:27:26,513 INFO mapred.MapTask: bufstart = 0; bufend = 96; bufvoid = 104857600
2026-04-23 05:27:26,513 INFO mapred.MapTask: kvstart = 26214396(104857584); kvend = 26214384(1048575
2026-04-23 05:27:26,519 INFO mapred.MapTask: Finished spill 0
2026-04-23 05:27:26,531 INFO mapred.Task: Task:attempt_local904016510_0001_m_000000_0 is done. And i
2026-04-23 05:27:26,536 INFO mapred.LocalJobRunner: Records R/W=5/1
```

Why the earlier notebook failed

The earlier failures came from a few concrete problems:

1. Hadoop was not installed correctly in the first version, which caused `hadoop: command not found`. □filecite□turn2file0□
2. In the later version, the streaming JAR path handling was still wrong, which led to `Usage: hadoop jar <jar> [mainClass] args...` instead of running the job. □filecite□turn3file0□
3. `part-00000` is an **output file**, not an input. It only appears after a successful job run. □filecite□turn2file0□turn3file0□

This notebook fixes those issues by using:

- explicit JAR discovery
- a single resolved JAR path
- Python `subprocess`
- local Hadoop mode with `file:///...`